#1 Method `leftRotate` takes a `Treap` node `x` and performs a left rotation. It returns nothing but it has side effects on the treap.
#2 Check if `x` is null or the root of the tree: if it is so, there is an error.
#3 Use a convenience variable for `x`'s parent. Since `x` is not the root of the tree, `y` won't be `null`.
#4 We can only perform a left rotation on a right child: so, if `x` is `y`'s left child, there is an error.
#5 Use a convenience variable for `y`'s parent.
#6 We don't know if `p` is `null` or not, so we need to check.
#7 Once we know `p` is not `null`, we still don't know if `y` is its left or right children.
#8 If there is a parent for `y`, we need to update it, replacing `y` with `x`.
#9 If, instead, `p` is `null`, it means that `y` was the treap's root, and so we need to update the tree making `x` its new root.
#10 Now that a reference to `x` is stored (either as `p`'s child or as the new root), we can update `y`'s right subtree, that will now point to `x`'s former left subtree.
#11 Finally, we reconnect `y` to the tree by setting it as `x`'s new right child.

First, let's assume we call rotate on node `X`, and node `Y` is `X`'s parent; once again, we analyze right rotation, so `X` is a left child of `Y`.

If we consider the subtree rooted at `Y`, we can visually "rotate" it clockwise (hence "right rotation"), pivoting on node `X`, until `X` looks like the root of this tree (hence, all other nodes appears to be below `X`).

The result should look something like the top-right quadrant of figure 3.6: of course, in order for this to be a valid BST rooted at node `X`, we need to make a few changes. For instance, there